

Lab Program 2

Develop a multistage dockerfile for container Orchestration

Name : Harshavardhan B R

USN : 1RV24MC042

Step 1 : Directory and file creation

Create the all the necessary files as shown below

```
> tree -I node_modules
.
├── dist
├── Dockerfile
├── package.json
├── package-lock.json
└── src
    └── index.js

3 directories, 4 files
> ls
dist  Dockerfile  node_modules  package.json  package-lock.json  src
```

Step 2 : Add necessary content

```
//src/index.js
const express = require("express")
const app = express()
const PORT = 3000

app.get('/', (req, res) => {
  res.send('Hello from multi stage docker ')
})

app.listen(PORT, () => {
  console.log(`server running on port ${PORT}`);
})
```

Add build script in package.json

```
//package.json
{
  "name": "l3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "build": "mkdir -p dist && cp -r src/* dist/"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Step 3 : Create Dockerfile

```
#Dockerfile
FROM node:20-alpine AS builder

WORKDIR /app
```

```

COPY package.json package-lock.json ./

RUN npm install

COPY . .

RUN npm run build

FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package.json ./

COPY --from=builder /app/package-lock.json ./

COPY --from=builder /app/dist ./dist

COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000

CMD [ "node", "dist/index.js" ]

```

Step 4 : Build an image

```
> docker build -t secondprog .
```

```

$ docker build -t secondprog .
[+] Building 2.4s (16/16) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 427B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  2.3s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df28 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 43.51kB                                  0.0s
=> CACHED [builder 2/6] WORKDIR /app                               0.0s
=> CACHED [builder 3/6] COPY package.json package-lock.json ./    0.0s
=> CACHED [builder 4/6] RUN npm install                             0.0s
=> CACHED [builder 5/6] COPY . .                                    0.0s
=> CACHED [builder 6/6] RUN npm run build                           0.0s
=> CACHED [stage-1 3/6] COPY --from=builder /app/package.json ./  0.0s
=> CACHED [stage-1 4/6] COPY --from=builder /app/package-lock.json ./ 0.0s
=> CACHED [stage-1 5/6] COPY --from=builder /app/dist ./dist      0.0s
=> CACHED [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules 0.0s
=> exporting to image                                              0.0s
=> => exporting layers                                              0.0s
=> => writing image sha256:457acbbe23b3bd563c2cd57b32dfa056eaaea448417bf9144e913688 0.0s
=> => naming to docker.io/library/secondprog                       0.0s

```

```

$ docker images | grep secondprog
secondprog          latest              457acbbe23b      32 minutes ago    137MB

```

Step 5: Run and verify

```

Run 'docker run --help' for more information
$ docker run -p 5500:3000 secondprog
server running on port 3000

```

```

$ curl -X GET localhost:3000
curl: (56) Recv failure: Connection reset by peer
$ curl -X GET localhost:5500
Hello from multi stage docker

```